

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 07 · LECTURE NOTES

Classical Planning

Search where the actions are described, not enumerated.

MODULE OUTCOMES

- Write a planning domain in terms of preconditions and effects.
- Explain why a declarative action description enables automatic heuristics.
- Derive the delete-relaxation heuristics.
- Explain goal-oriented action planning and why games use it.
- State the limits of classical planning.

1 Declarative action descriptions

```
STATE: a set of facts taken to be true.  
      { At(Agent, Room1), Has(Key), Closed(Door12) }
```

```
ACTION SCHEMA:  
  Open(d, r1, r2)  
    PRECOND: At(Agent,r1), Connects(d,r1,r2),  
             Closed(d), Has(Key)  
    ADD:     Open(d)  
    DELETE:  Closed(d)
```

A PLAN is a sequence of actions leading from the initial state to a state satisfying the goal. This is still Module 02's search: states, a successor function (apply an action whose preconditions hold), and a goal test.

THE DIFFERENCE IS THAT THE ACTIONS ARE DESCRIBED RATHER THAN CODED. And that single change is what makes everything else in this module possible:

- the planner can REASON ABOUT the actions themselves
- it can compute which actions are RELEVANT to a goal
- it can detect that two actions are INDEPENDENT and so can be reordered or parallelised
- and it can DERIVE A HEURISTIC AUTOMATICALLY (section 2)

THAT LAST ONE IS THE PAYOFF. With a hand-coded successor function you must invent a heuristic yourself (Module 03 section 3). With a declarative domain the planner derives one from the action descriptions, with no domain-specific insight whatsoever.

Declarativeness buys automatic heuristics, and that is why planning exists as a separate field rather than as an application of Module 02. It is worth sitting with the point: **the reason to write the domain in a restricted, inspectable language is to let the solver do work that would otherwise require you** — which is the same argument as CSCE 669 Module 13's case for a modelling layer, and the same argument as declarative query languages against hand-written loops (CSCE 608).

2 Heuristics for free

IGNORE THE DELETE LISTS, AND THE PROBLEM BECOMES EASY

Drop every action's delete list. Facts can then only accumulate — nothing that becomes true ever becomes false again, so there is no possibility of undoing progress.

The relaxed problem is solvable in polynomial time by forward chaining to a fixed point, **and its cost is an admissible heuristic for the original problem**, because every real plan is also a valid plan in the relaxed problem and therefore the relaxed optimum is a lower bound.

This is Module 03 §3's relaxation argument applied automatically. The planner derives the relaxation from the domain description, mechanically, with no human insight — which is why this result transformed automated planning from a field that could solve toy problems into one that solves large ones. **And it is CSCE 669 Module 10 §1's bound-from-a-relaxation mechanism for the third time in this program**, which at this point should be read as a general technique rather than a coincidence.

Three heuristics follow from it: $h_{\max}$ (the cost of the most expensive individual subgoal — admissible and weak), **h_{add}** (the sum of subgoal costs — inadmissible, because it double-counts work shared between subgoals, and stronger), and **h_{FF}** (extract an actual relaxed plan and count its actions — inadmissible, strong, cheap, and the heuristic that made planners work in practice).

Heuristic	What it computes	Property
$h_{\max}$	The cost of the single most expensive subgoal in the relaxation.	Admissible, and weak — it ignores that the other subgoals also cost something.
h_{add}	The sum of all subgoal costs.	Inadmissible, because work shared between subgoals is counted more than once; substantially stronger in practice.
h_{FF}	Extract an actual plan for the relaxed problem and count its actions.	Inadmissible, strong, and cheap — the practical winner, and it also yields <i>helpful actions</i> (the first actions of the relaxed plan) which prune the branching enormously.
Landmark heuristics	Facts or actions that every valid plan must achieve.	Admissible variants exist and are strong, and they are the basis of most current optimal planners.

Modern planners mostly use inadmissible heuristics with greedy best-first search, trading optimality for the ability to solve the problem at all — which is worth stating plainly, since Module 03 spent its effort on admissibility. **The honest position is that optimal planning is a much smaller field than satisficing**

planning, and for most applications (including every game application) a good plan found quickly beats an optimal plan found slowly.

3 Planning in games

GOAP: PLAN BACKWARDS FROM A GOAL THROUGH ACTION PRECONDITIONS

Give each action a set of preconditions and effects, give the agent a goal, and search backwards from the goal through actions whose effects satisfy it, until reaching the current world state. That is goal-oriented action planning, and backwards search is used because the goal is specific and the state space is large.

The appeal is authoring. Add a new action and every agent can use it immediately, in combinations nobody anticipated — there is no state machine to rewire and no transitions to add (Module 11 §1's comparison). The authoring cost is linear in actions rather than quadratic in states, which is the concrete form of the advantage.

Which produces emergent, situation-appropriate behaviour. An agent that takes cover and then reloads, because the plan required both ammunition and safety and the planner ordered them sensibly — not because anyone scripted 'reload behind cover'. This is what made GOAP notable when it shipped.

And the cost is debuggability. When the agent does something strange, the cause is an interaction between action descriptions, which is far harder to locate than a bad transition in a state machine — and it may only manifest in one world configuration. Which is why many studios moved back to behaviour trees (Module 11 §2), and is a genuine engineering trade rather than a failure of the technique: the combinatorial authoring that makes GOAP powerful is exactly what makes it hard to reason about.

4 Limits

- **Deterministic actions.** If an action can fail, a sequence is not enough — **you need a policy that says what to do in each resulting state** (Module 10), or **replanning on failure**, which is what games actually do.
- **Full observability.** Otherwise you must plan over belief states rather than states, and the space explodes (Module 01 §2).
- **A static world. Nothing else changes while you act** — which is false in any game containing other agents, and is the assumption most obviously violated.
- **Instantaneous, non-overlapping actions.** Real actions take time and can overlap, which requires temporal planning with durations and concurrency — a substantially harder problem.
- **And discrete facts.** Numeric resources — fuel, ammunition counts, health — require numeric planning, which is harder again and where planning starts to overlap with CSCE 669's optimisation.
- **Games survive these violations by replanning frequently:** a plan is treated as a *current intention* rather than a commitment, discarded as soon as the world invalidates it, and recomputed. **Which is Module 04 §3's replanning discipline applied to behaviour rather than to paths**, and it is the practical resolution of all five assumptions at once.

HIERARCHICAL PLANNING IS HOW THE SCALE PROBLEM IS HANDLED

Hierarchical task networks decompose a high-level task into *methods*, each a partially ordered set of subtasks, recursively down to primitive actions — so the search is over decompositions rather than over action sequences.

So the domain author supplies the decomposition knowledge rather than relying on search to discover a long plan from scratch — **which is why HTN planners solve substantially larger problems than classical ones** and are what most industrial planning applications actually use.

The trade is that you are back to authoring structure, so you lose part of §3's combinatorial-authoring benefit — adding an action no longer automatically makes it available everywhere, because it must appear in some method.

And this is Module 02 §4's abstraction arriving again — the same reduction in effective search depth, with **the hierarchy supplied by a person rather than derived from the problem. Which is the recurring trade of this whole course:** human knowledge reduces search, search reduces the need for human knowledge, and every practical system picks a point between them.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Russell & Norvig — AIMA, chapters 11–12 (pseudocode free)	STRIPS, the delete relaxation, and the heuristics of §2 , developed carefully.
Hoffmann & Nebel — The FF Planning System (free)	The h_FF heuristic and helpful actions of §2 , which is the paper that changed the field's capability.
Orkin — Three States and a Plan: The AI of F.E.A.R. (free)	The §3 GOAP case study , by the person who shipped it. The clearest account of both the appeal and the cost.
Ghallab, Nau & Traverso — Automated Planning and Acting (free draft)	The reference for §4 , including HTN planning and the treatment of acting under the violated assumptions.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write a STRIPS domain** for a small problem with at least six action schemas.
- 2 **Implement forward search over it** with $h = 0$ and report node counts.
- 3 **Implement the delete relaxation** and compute $h_{\max}$ and h_{add} .

- 4 **Compare node counts** with each heuristic and confirm h_{add} is stronger and inadmissible.
- 5 **Implement h_{FF}** by extracting a relaxed plan, and report the improvement.
- 6 **Extract helpful actions** from the relaxed plan and use them to prune. Report the effect.
- 7 **Build a small GOAP agent** with eight actions and observe which plans it produces.
- 8 **Add one new action** and report which new behaviours appeared without further authoring.
- 9 **Then find a plan it produces that you consider wrong**, and trace it to the action interaction responsible.
- 10 **Make an action fail at runtime** and implement replanning. Report how often it triggers.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write an action in STRIPS form and define a plan.
- 2 What does declarative action description buy, and why is that the whole point?
- 3 Explain the delete relaxation and why its cost is admissible.
- 4 Compare h_{max} , h_{add} , h_{FF} , and landmarks.
- 5 Why do modern planners mostly use inadmissible heuristics?
- 6 What is GOAP's authoring advantage, stated precisely?
- 7 What is GOAP's cost, and why did studios move away from it?
- 8 Name five classical assumptions and what fails without each.
- 9 How do games survive those violations?
- 10 What does HTN planning trade, and what does it correspond to?